UNIVERSIDAD DEL VALLE

FACULTAD DE INFORMÁTICA Y ELECTRÓNICA

CARRERA DE INGENIERÍA DE SISTEMAS INFORMÁTICOS

LG15 – Tolerancia a fallos y replicación de datos en HDFS

ESTUDIANTES:	Edson Marcelo Cayo Ali
	Joel Julio Salazar Ferrufino
	Alex Bryam Barrera Alarcon
MATERIA:	Tecnologías Emergentes I
GRUPO:	B
CÓDIGO DE PRÁCTICA	LG15
DOCENTE:	Ing. Efrain Luna Mamani

Cochabamba - Bolivia

Gestión 2 – 2026

1. Descripción del proyecto

Se implementó un entorno distribuido de Hadoop HDFS utilizando Docker Compose, con el objetivo de evaluar la tolerancia a fallos y la replicación de datos.

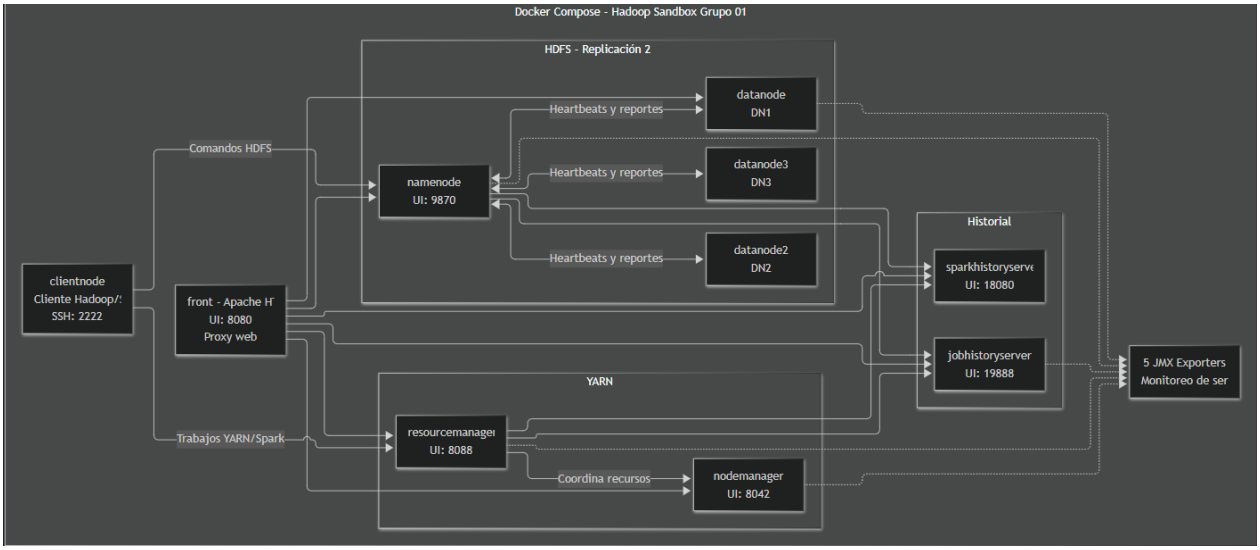
Se realizaron dos experimentos utilizando datasets de 1 GB y 10 GB, simulando la caída de un DataNode y verificando la disponibilidad de los archivos, la distribución de los bloques y su recuperación.

Arquitectura del clúster:

Componente	Configuración
NameNode	1
DataNodes	3
Factor de replicación	2
Datasets	1 GB y 10 GB
Tecnología	Hadoop HDFS y Docker Compose

El NameNode administra los metadatos y las ubicaciones de los bloques, mientras que los tres DataNodes almacenan los datos y sus réplicas.

1.1 Diagrama de arquitectura

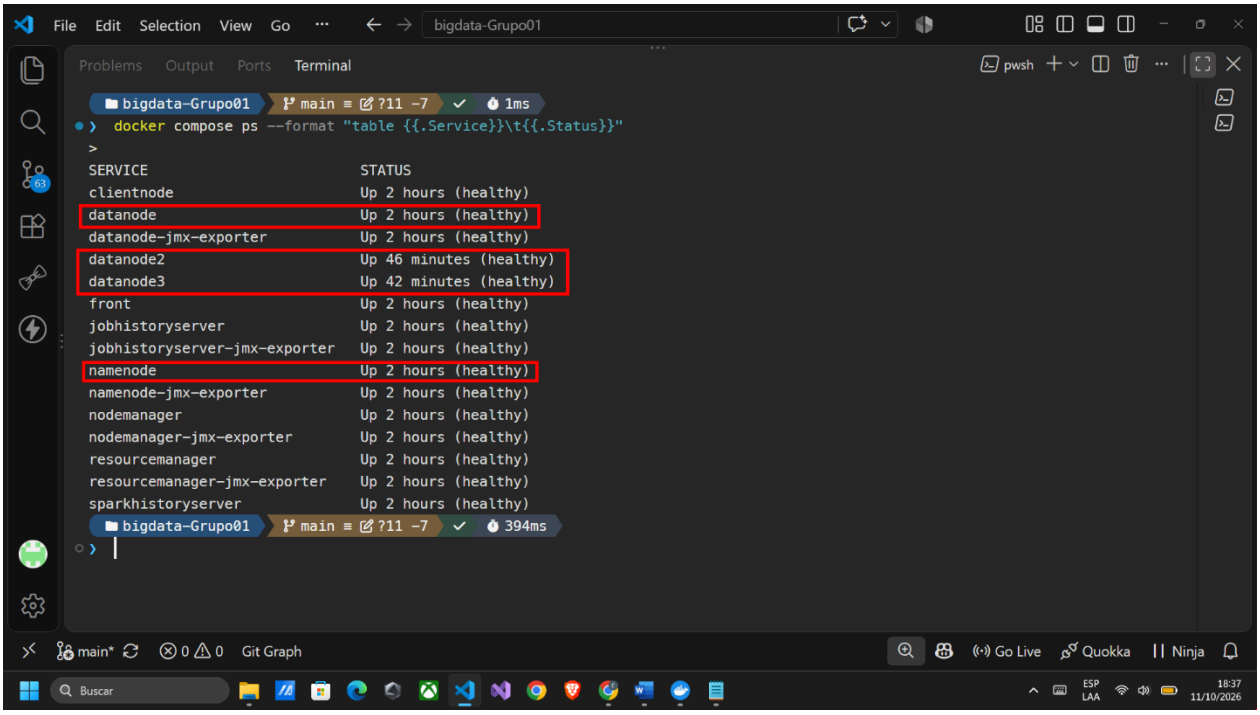


2. Instalación y preparación del entorno

2.1. Instalación y ejecución

Se utilizó Docker Compose para desplegar el clúster Hadoop HDFS, compuesto por un NameNode y tres DataNodes.

- git clone https://github.com/juliosalazar98/bigdata-Grupo01.git
- cd bigdata-Grupo01
- docker compose up -d



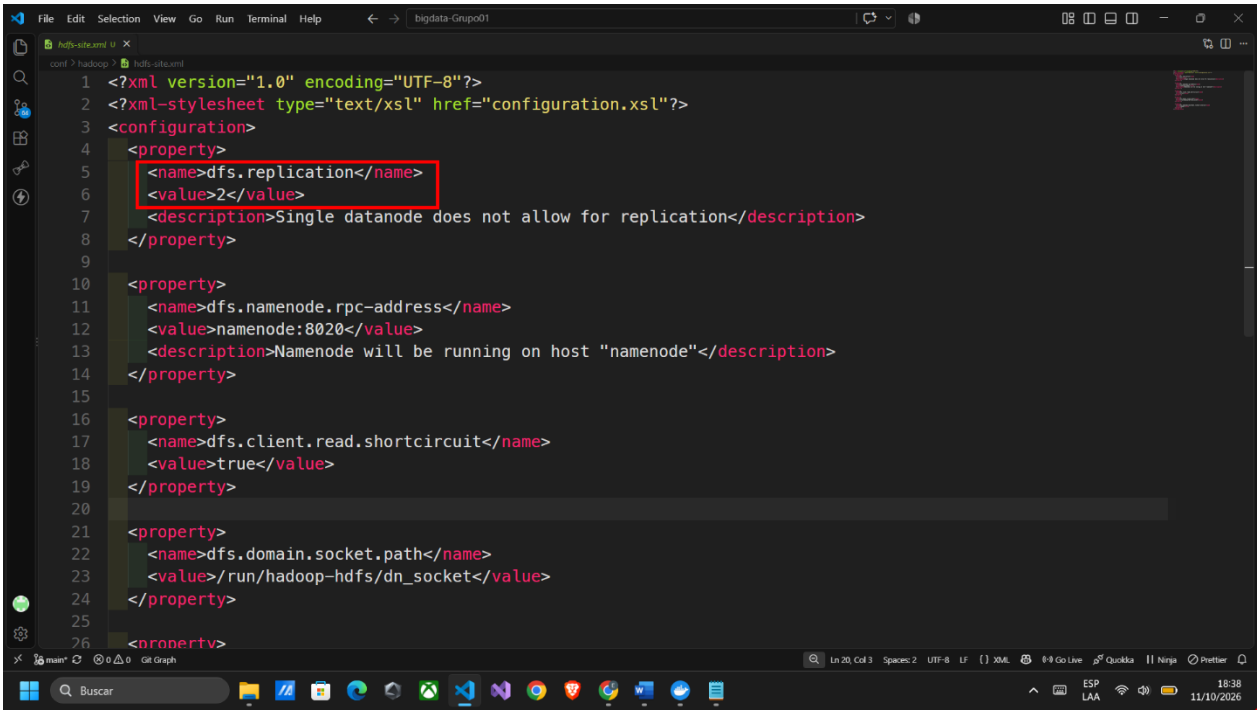
Se verificó el estado de los DataNodes y se creó el directorio correspondiente al grupo:

- docker compose exec -u hdfs namenode hdfs dfsadmin -report
- docker compose exec -u hdfs namenode hdfs dfs -mkdir -p /grupo01
- docker compose exec -u hdfs namenode hdfs dfs -chown sandbox:sandbox /grupo01

2.2. Configuración del clúster HDFS

Se modificaron los archivos de configuración para cumplir con los requisitos de la práctica:

- **docker-compose.yaml:** se agregaron dos servicios DataNode adicionales, obteniendo un total de tres DataNodes y un NameNode.
- **hdfs-site.xml:** se estableció el parámetro dfs.replication con valor 2, permitiendo almacenar dos réplicas de cada bloque en diferentes DataNodes.



Posteriormente, se verificó el factor de replicación desde el nodo cliente:

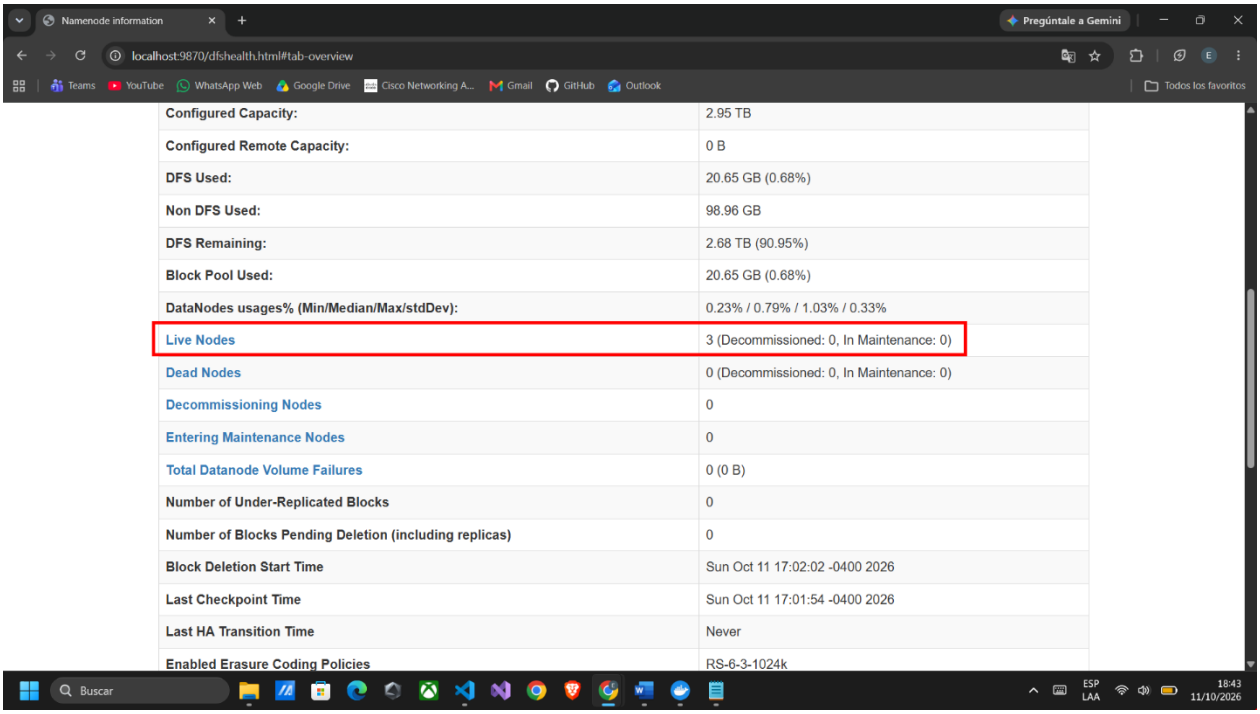
- hdfs getconf -confKey dfs.replication

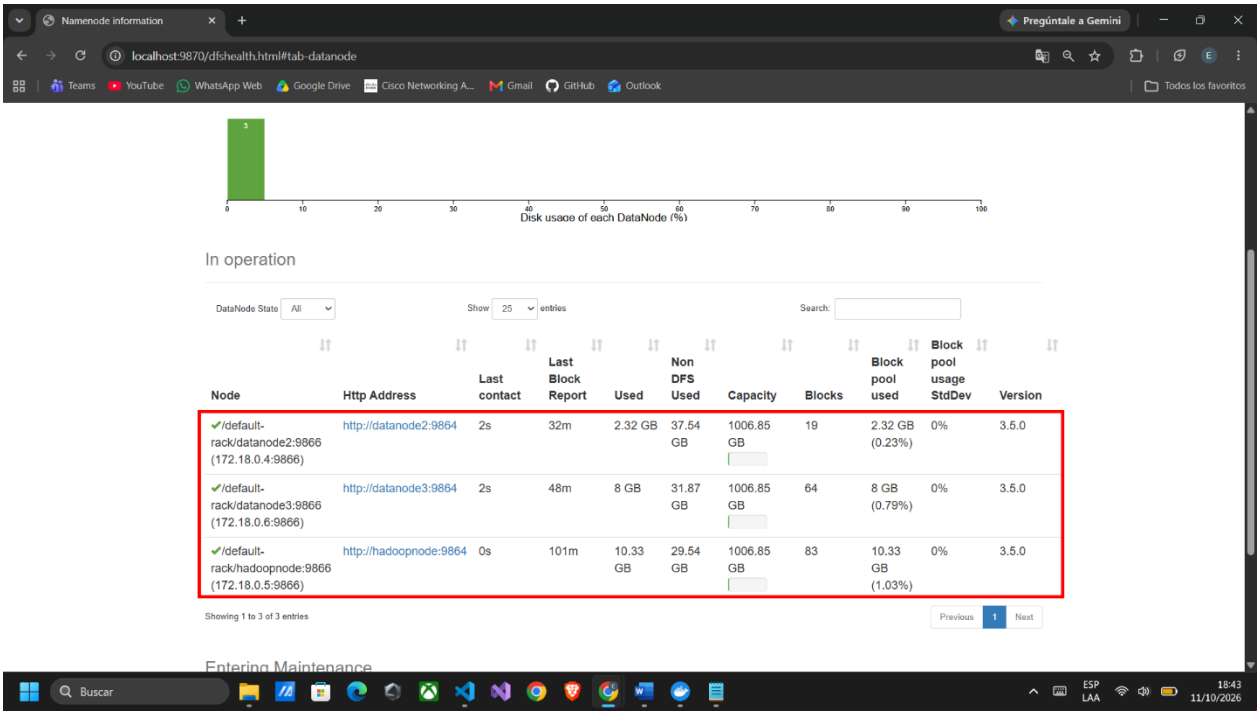
El comando devolvió 2, confirmando la configuración utilizada para las pruebas.

También se verificó el estado de los tres DataNodes mediante:

- docker compose exec -u hdfs namenode hdfs dfsadmin -report

Esto permitió comprobar que el clúster contaba con los nodos necesarios para realizar las pruebas de tolerancia a fallos.





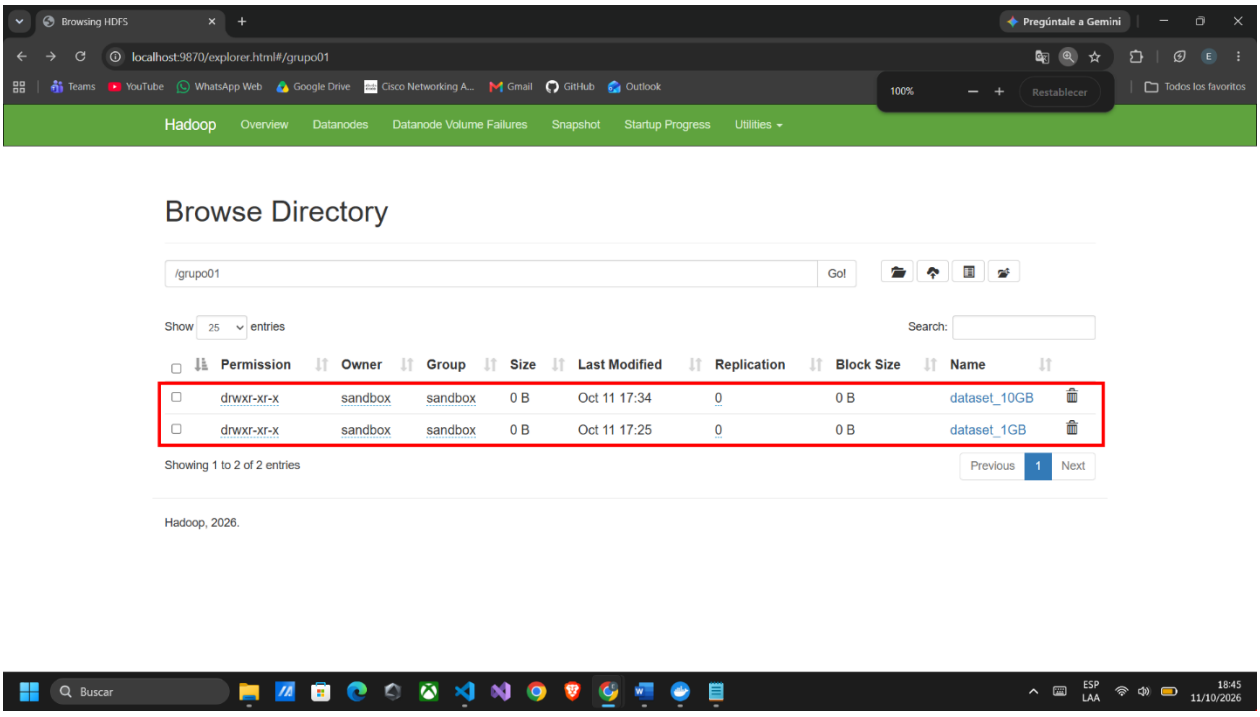
2.3. Preparación de los datasets

Se ingresó al nodo cliente:

- docker compose exec -u sandbox clientnode bash

Dentro del nodo se verificó el factor de replicación y se prepararon las rutas:

- hdfs getconf -confKey dfs.replication
- BASE=/grupo01
- F1="$BASE/dataset_1GB/dataset_grupo01_1GB"
- F10="$BASE/dataset_10GB/dataset_grupo01_10GB"
- hdfs dfs -mkdir -p "$BASE/dataset_1GB" "$BASE/dataset_10GB"
- hdfs dfs -ls "$BASE"



El factor de replicación configurado fue 2, con un tamaño de bloque de 128 MiB.

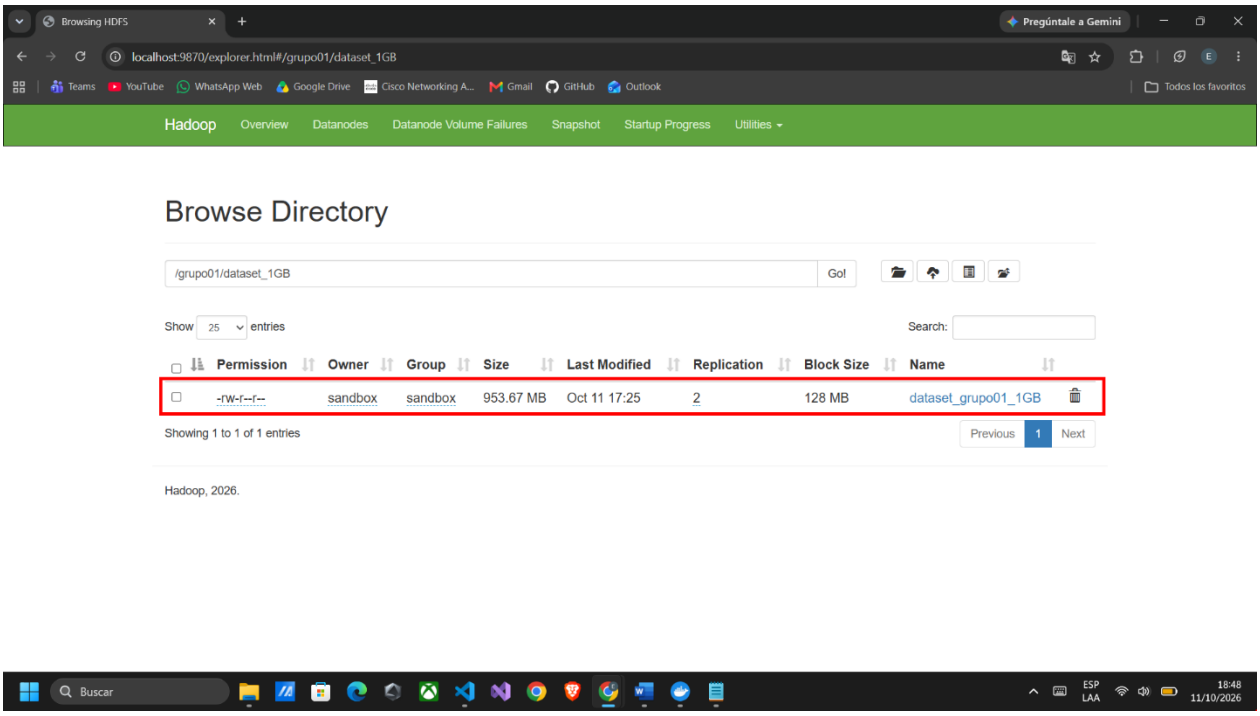
3. Pruebas de fallo con cada dataset

3.1. Experimento con dataset de 1 GB

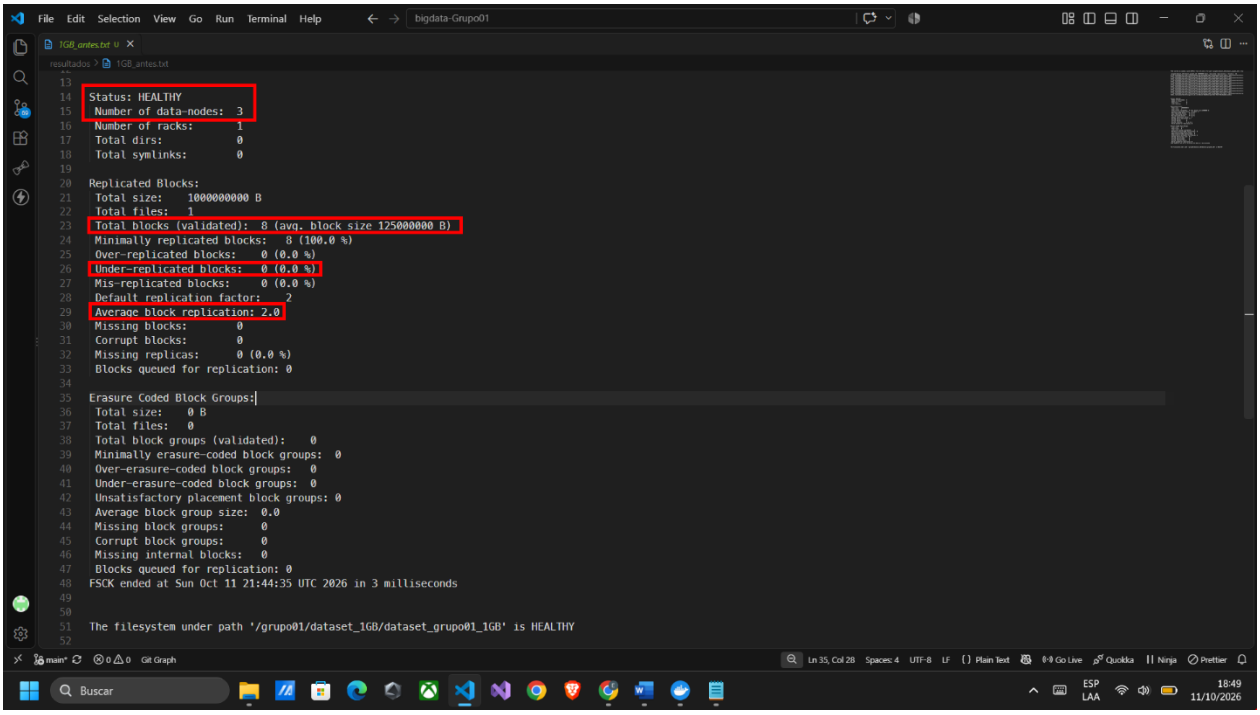
Carga y verificación inicial

Se generó un archivo de 1.000.000.000 bytes utilizando /dev/zero y se almacenó en HDFS.

- head -c 1000000000 /dev/zero | hdfs dfs -put - "$F1"
- hdfs dfs -ls -h "$F1"
- hdfs dfs -stat 'Tamaño: %b bytes | Replicacion: %r' "$F1"
- hdfs fsck "$F1" -files -blocks -locations



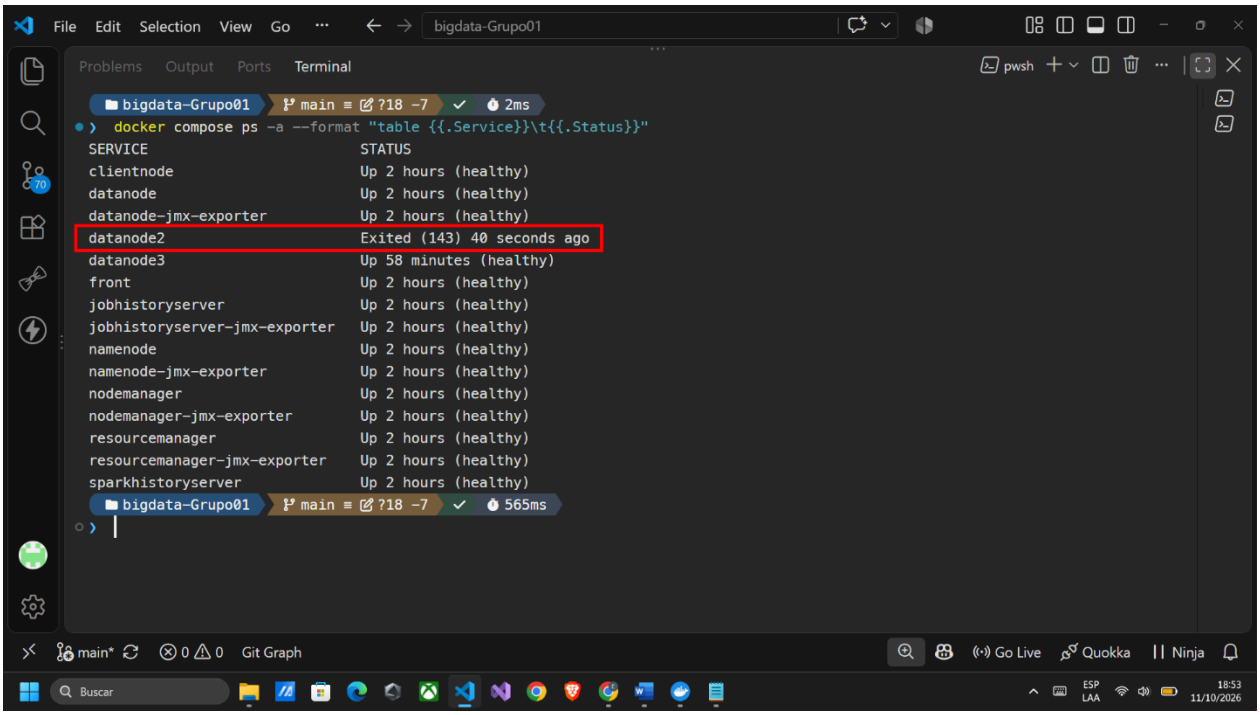
Inicialmente se registraron tres DataNodes activos. El archivo contenía 8 bloques, cada uno con dos réplicas, distribuidas entre los tres nodos. El estado reportado fue HEALTHY.



Simulación de fallo

Desde la terminal del host se detuvo únicamente el DataNode 2:

- docker compose stop datanode2



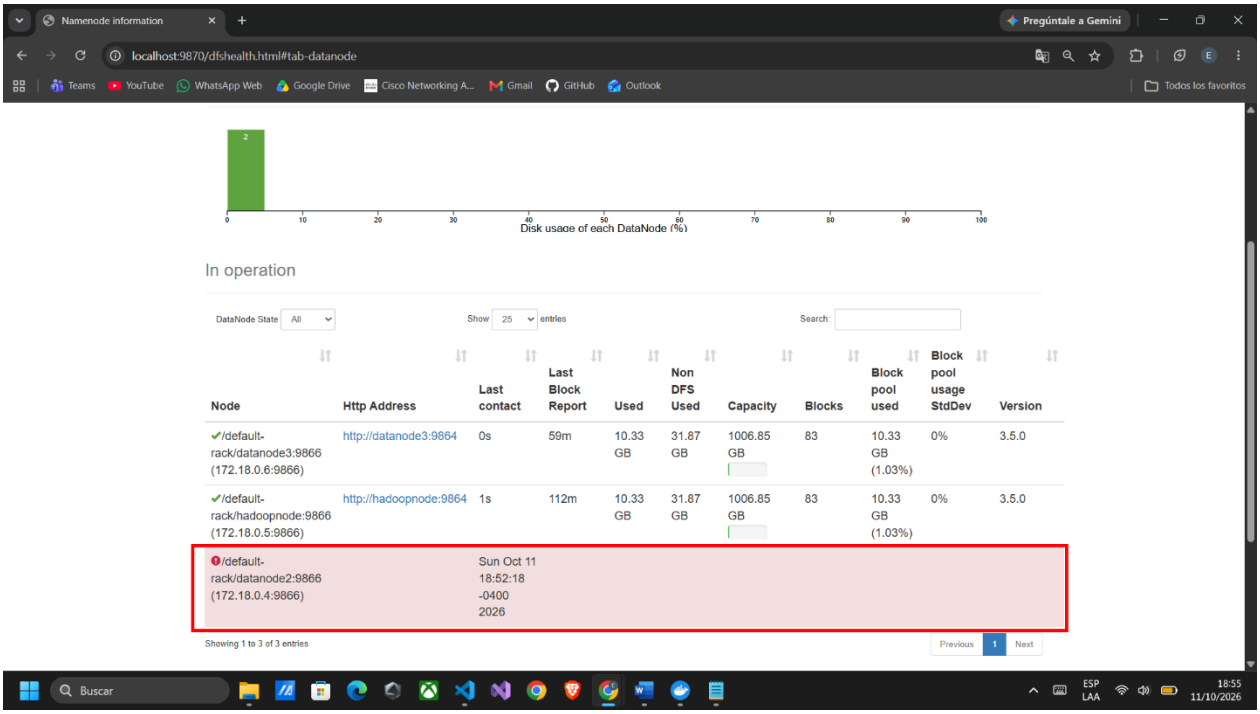
En el nodo cliente se verificó la disponibilidad y lectura del archivo:

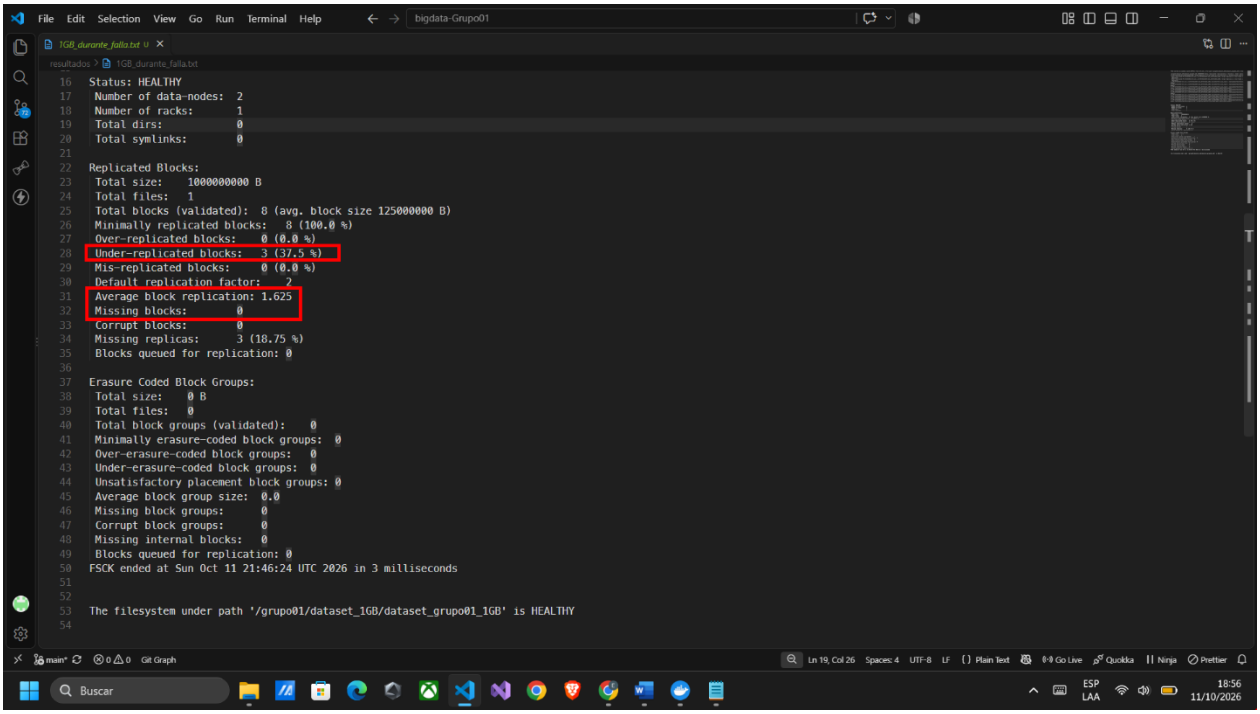
- hdfs dfs -test -e "$F1" && echo "Archivo disponible"
- hdfs dfs -cat "$F1" > /dev/null && echo "Lectura correcta"
- hdfs fsck "$F1" -files -blocks -locations

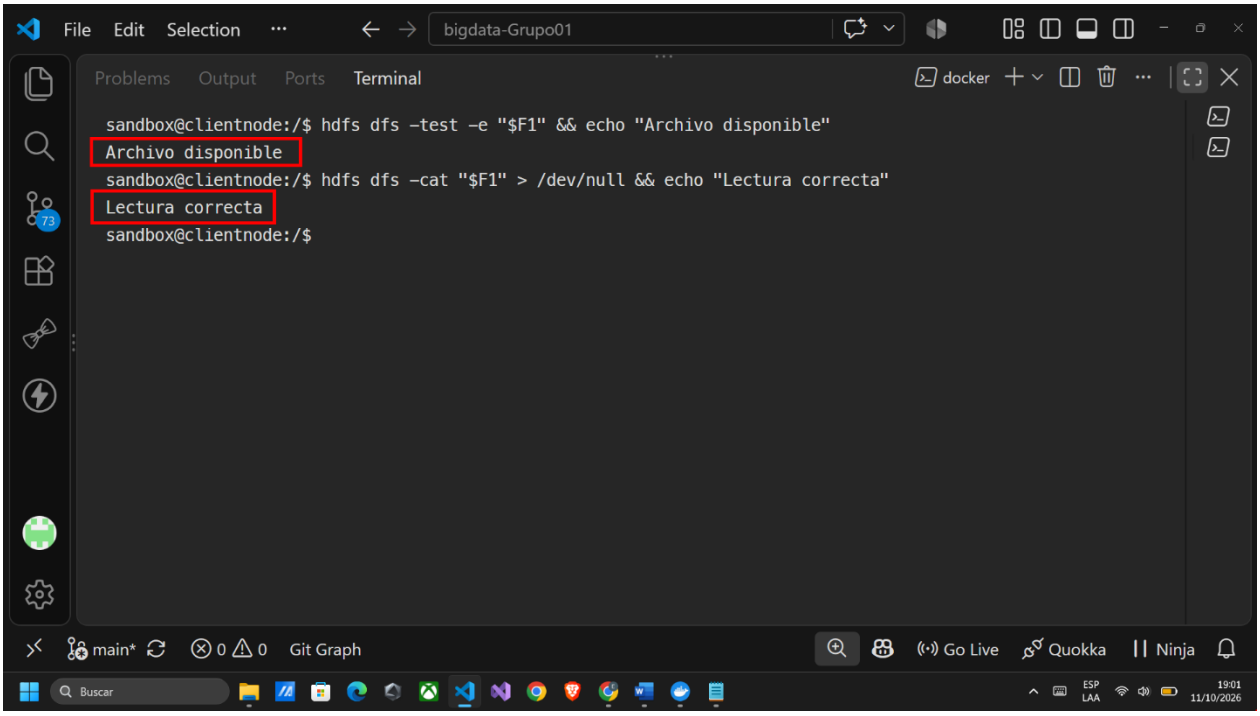
Resultados obtenidos

El DataNode 2 almacenaba inicialmente cuatro bloques del archivo. Después de detenerlo, HDFS reportó:

- 2 DataNodes activos.
- 8 bloques disponibles de 8.
- 3 bloques subreplicados (37,5 %), cada uno con una réplica activa.
- 0 bloques perdidos o corruptos.
- Estado del archivo: HEALTHY.
- Lectura del archivo: correcta.



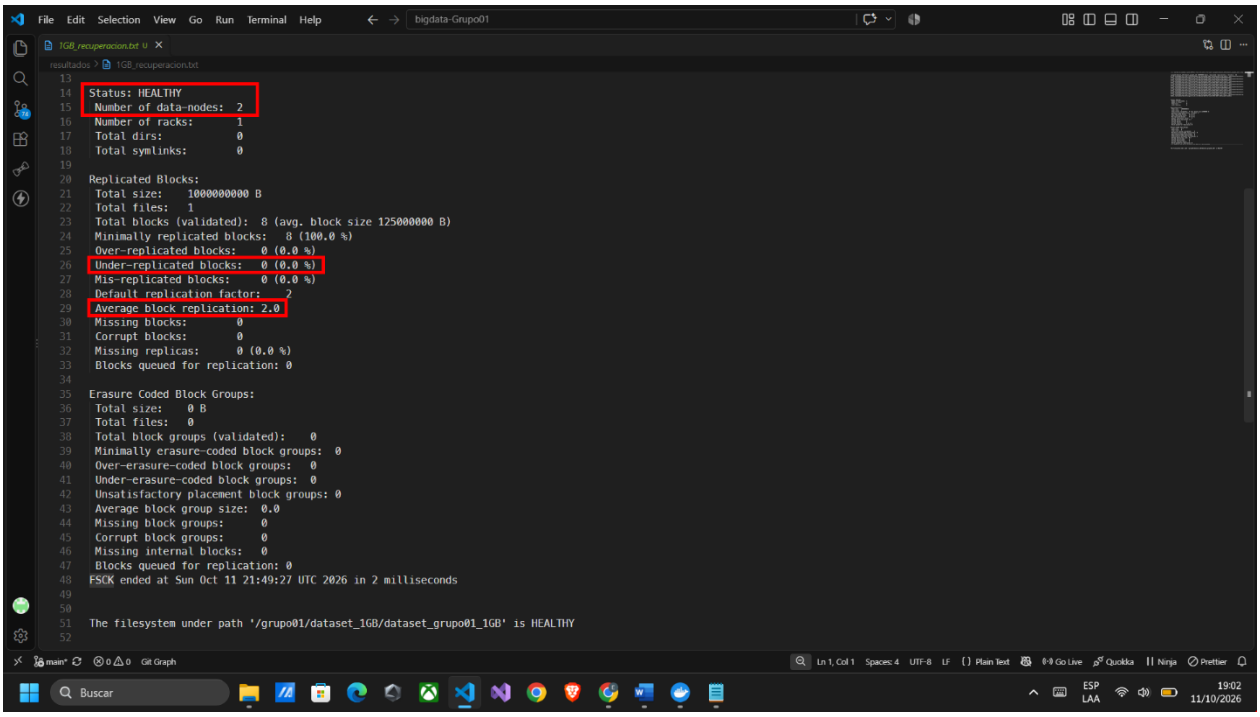




Uno de los cuatro bloques afectados ya había recuperado su segunda réplica cuando se realizó la verificación.

Recuperación

La verificación posterior mostró que los ocho bloques volvieron a tener dos réplicas, sin bloques subreplicados. Las nuevas copias se distribuyeron entre los DataNodes restantes, demostrando la recuperación automática de HDFS.



Finalmente, se reinició el nodo detenido:

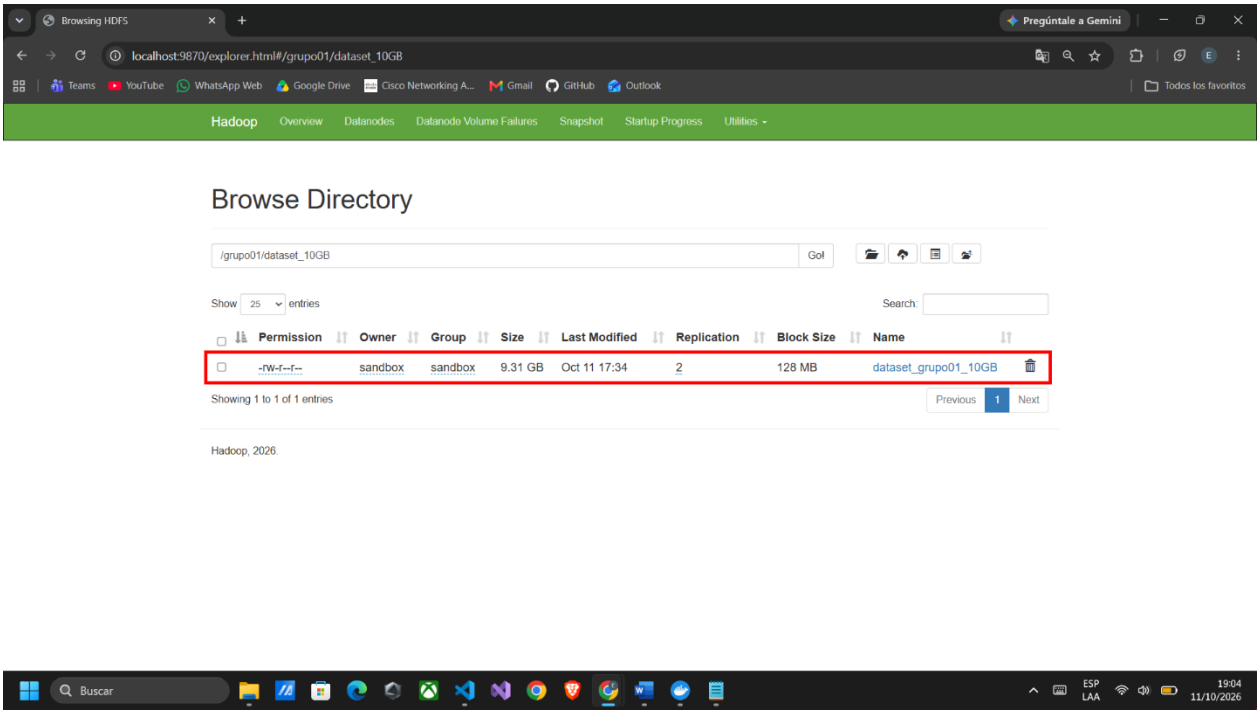
- docker compose start datanode2

3.2. Experimento con dataset de 10 GB

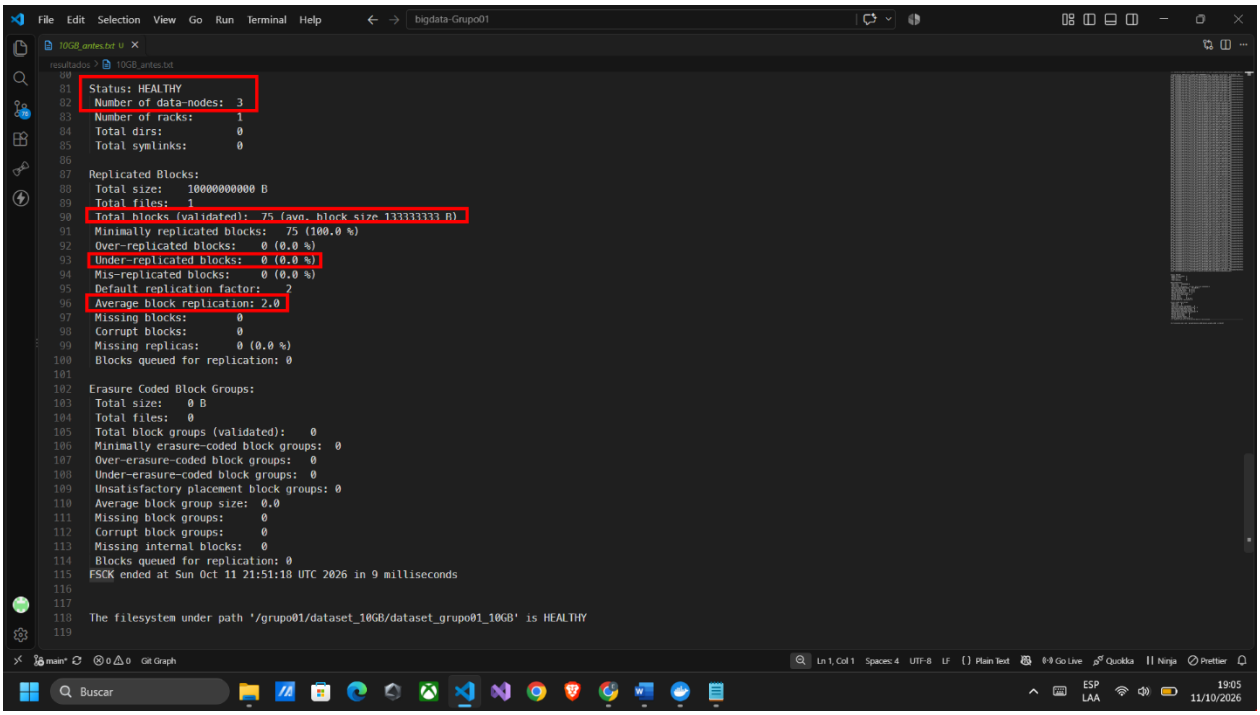
Carga y verificación inicial

Se generó un archivo de 10.000.000.000 bytes y se almacenó en HDFS.

- head -c 10000000000 /dev/zero | hdfs dfs -put - "$F10"
- hdfs dfs -ls -h "$F10"
- hdfs dfs -stat 'Tamaño: %b bytes | Replicacion: %r' "$F10"
- hdfs fsck "$F10" -files -blocks -locations



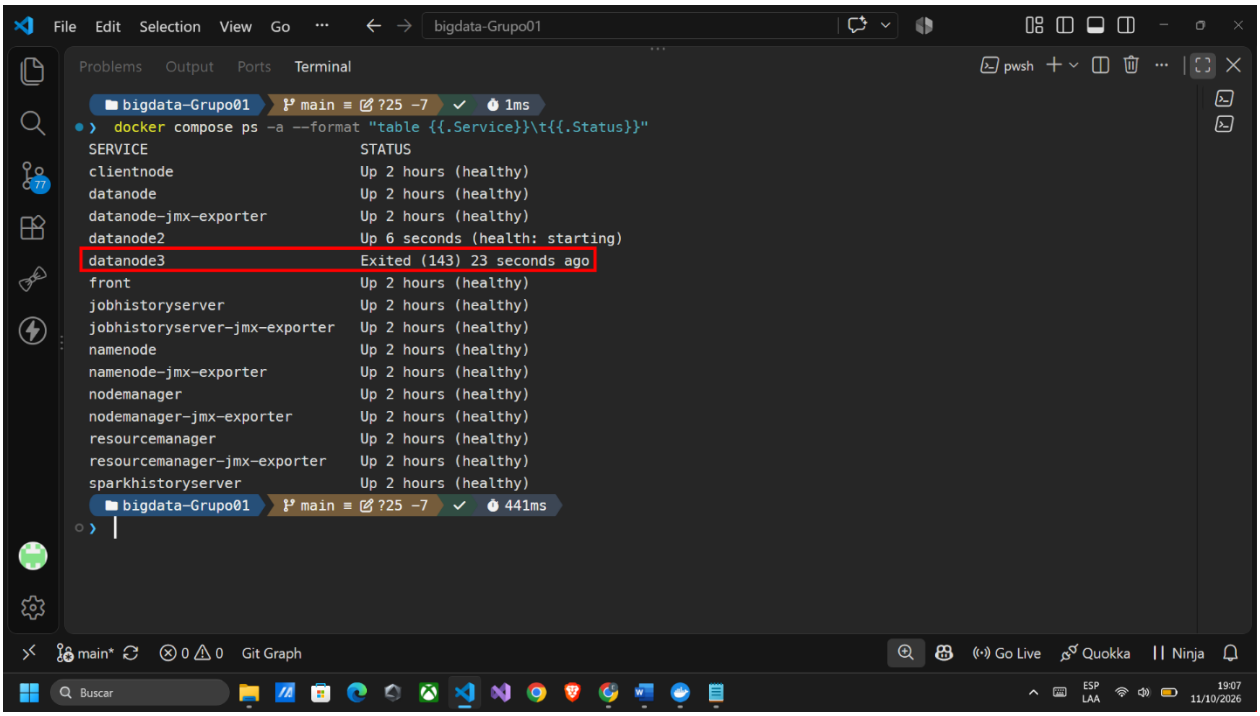
El archivo contenía 75 bloques, todos con dos réplicas distribuidas entre los tres DataNodes. Su estado inicial fue HEALTHY.



Simulación de fallo

Desde el host se detuvo el DataNode 3:

- docker compose stop datanode3



Posteriormente, en el nodo cliente se ejecutaron las verificaciones:

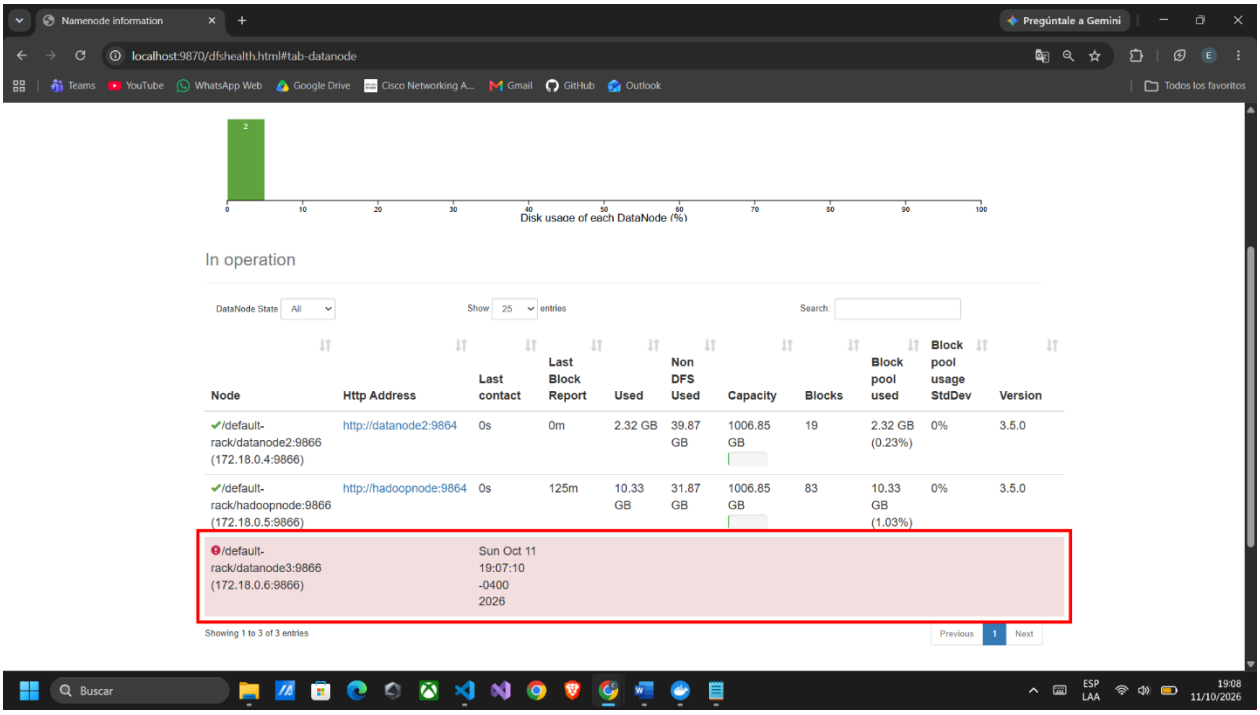
- hdfs dfs -test -e "$F10" && echo "Archivo disponible"
- hdfs dfs -cat "$F10" > /dev/null && echo "Lectura correcta"
- hdfs fsck "$F10" -files -blocks -locations

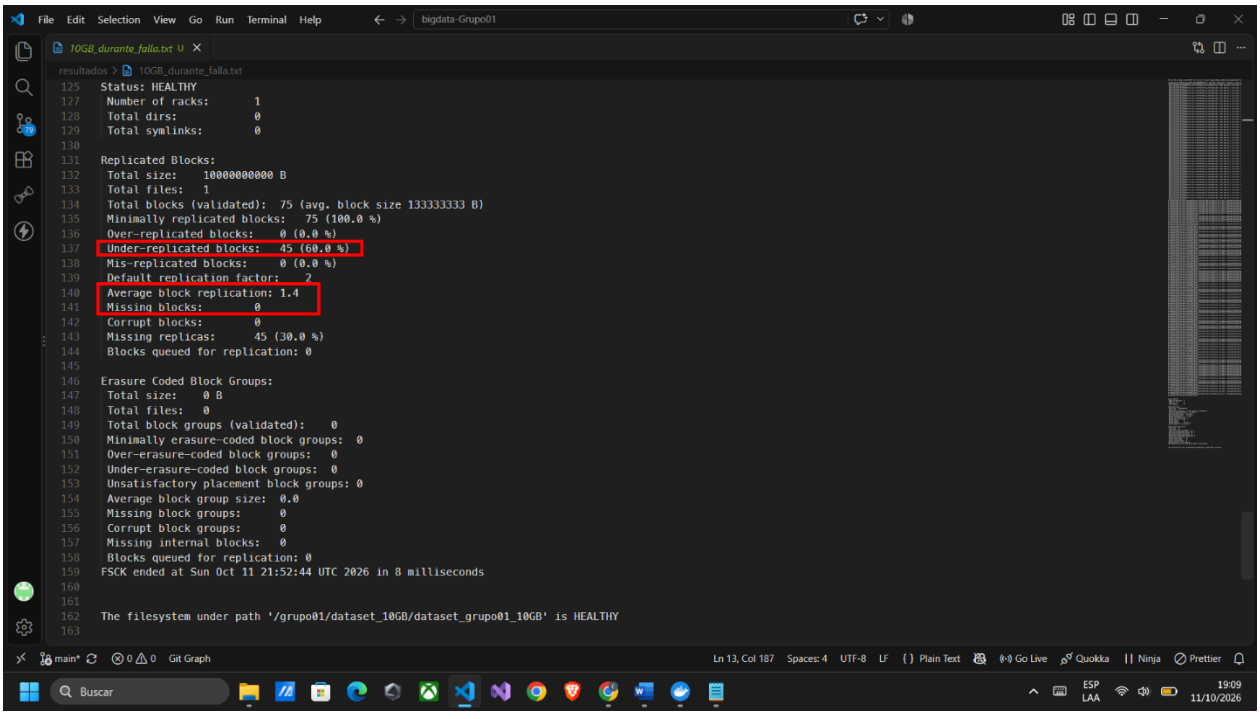
Resultados obtenidos

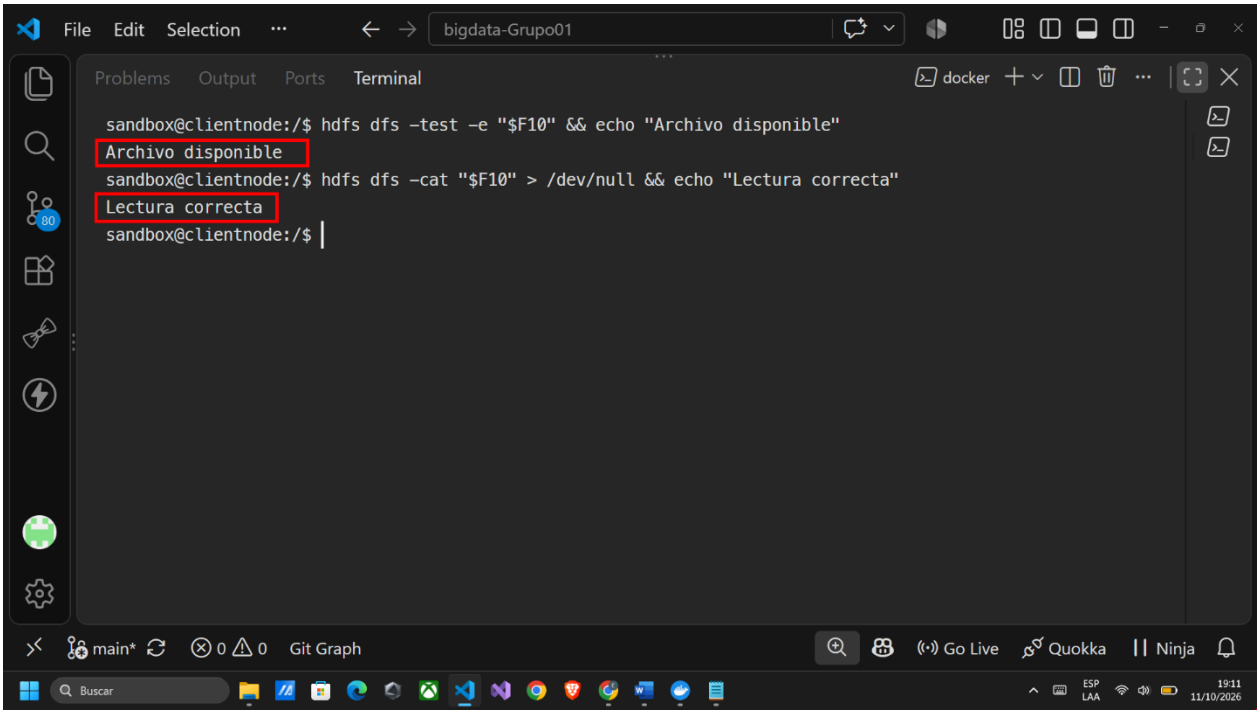
El DataNode 3 almacenaba inicialmente 57 bloques. Durante la verificación se obtuvo:

- 2 DataNodes activos.
- 75 bloques disponibles de 75.
- 45 bloques subreplicados (60 %), con una réplica activa cada uno.

- 0 bloques perdidos o corruptos.
- Estado del archivo: HEALTHY.
- Lectura del archivo: correcta.



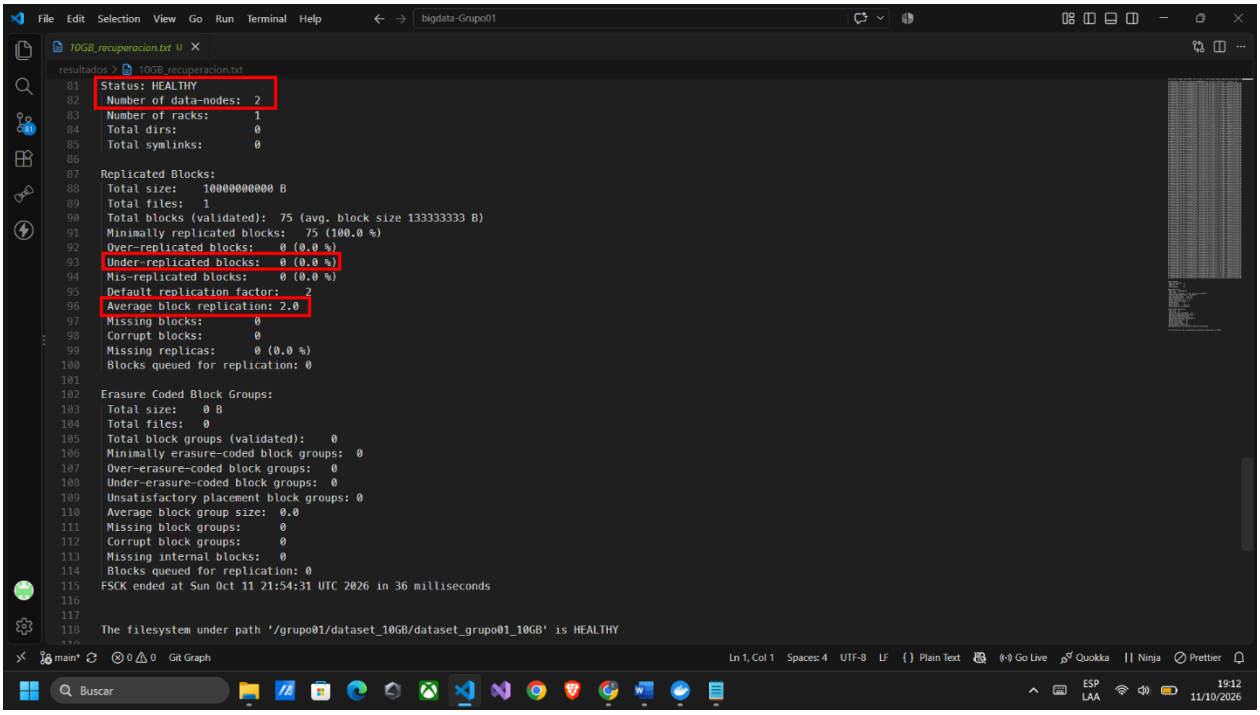




Los otros 12 bloques afectados ya habían recuperado su segunda réplica al momento de la comprobación.

Recuperación

En la verificación posterior, HDFS reportó 75 bloques con dos réplicas, sin bloques subreplicados. Las nuevas copias quedaron distribuidas entre los DataNodes 1 y 2.



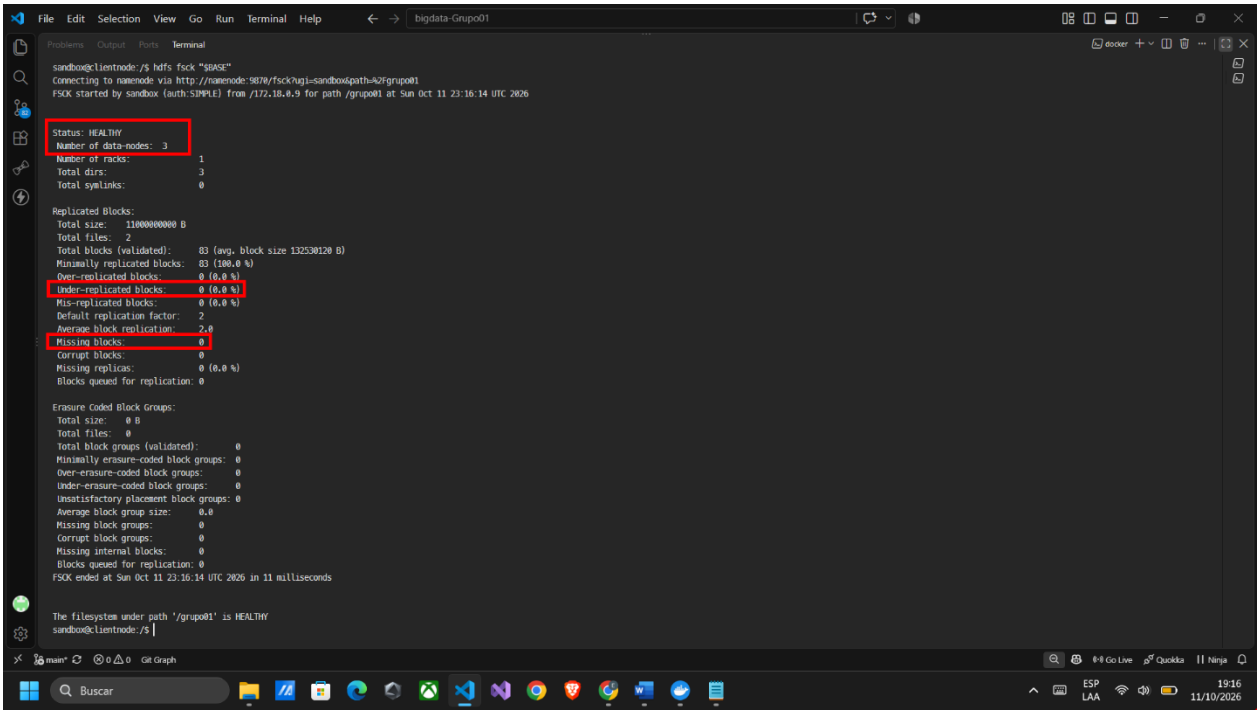
Se reinició el nodo detenido:

- docker compose start datanode3

3.3. Verificación final

Se utilizaron los siguientes comandos para comprobar la estructura de directorios, el espacio utilizado y la integridad de los archivos:

- hdfs dfs -ls -R "$BASE"
- hdfs dfs -du -h "$BASE"
- hdfs fsck "$BASE"



4. Análisis de resultados

4.1. Antes de la falla

Ambos datasets se encontraban en estado HEALTHY, con tres DataNodes activos y un factor de replicación de 2.

El archivo de 1 GB contenía 8 bloques, distribuidos en los tres DataNodes: DN1 almacenaba 7 bloques, DN2 almacenaba 4 y DN3 almacenaba 5.

El archivo de 10 GB contenía 75 bloques: DN1 almacenaba 42, DN2 almacenaba 51 y DN3 almacenaba 57. Cada bloque tenía dos copias en nodos diferentes.

4.2. Durante la falla

En la primera prueba se detuvo DN2, afectando inicialmente cuatro bloques del dataset de 1 GB. En la segunda se detuvo DN3, afectando inicialmente 57 bloques del dataset de 10 GB.

En ambos casos existían otras réplicas, por lo que los archivos continuaron disponibles y pudieron leerse completamente.

HDFS detectó una reducción temporal de las réplicas y reportó 3 bloques subreplicados en el dataset de 1 GB y 45 en el de 10 GB al momento de las verificaciones.

4.3. Después de la falla

En las verificaciones de recuperación, HDFS reportó estado HEALTHY, cero bloques subreplicados y un promedio de dos réplicas por bloque.

La recuperación se realizó automáticamente utilizando los DataNodes disponibles:

- **1 GB:** nuevas réplicas en DN1 y DN3.
- **10 GB:** nuevas réplicas en DN1 y DN2.

Esto ocurrió porque el NameNode supervisa la disponibilidad de los bloques y coordina la creación de nuevas copias cuando su cantidad es inferior al factor de replicación configurado.

Los registros demuestran que la recuperación ocurrió incluso mientras los DataNodes detenidos seguían fuera de servicio.

5. Comparación de resultados

Característica	Dataset 1 GB	Dataset 10 GB
Tamaño	1.000.000.000 bytes	10.000.000.000 bytes
Número de bloques	8	75
Tamaño de bloque	128 MiB	128 MiB
Factor de replicación	2	2

DataNode afectado	DN2 (172.18.0.4)	DN3 (172.18.0.6)
Bloques inicialmente afectados	4	57
Bloques subreplicados observados	3 (37,5 %)	45 (60 %)
Bloques disponibles después de la falla	8 de 8	75 de 75
¿Archivo accesible?	Sí	Sí
¿Lectura correcta?	Sí	Sí
¿Existieron bloques subreplicados?	Sí	Sí
Tiempo de recuperación		
Nuevas réplicas	4, en DN1 y DN3	57, en DN1 y DN2
Estado final	HEALTHY	HEALTHY

Interpretación: El dataset de 10 GB presentó un mayor número de bloques afectados y requirió generar más réplicas. Sin embargo, ambos experimentos demostraron que HDFS puede conservar la disponibilidad de los archivos y restablecer automáticamente la redundancia después de la falla de un DataNode.

6. Preguntas

1. ¿Por qué HDFS mantiene disponible el archivo cuando falla un DataNode?

Porque cada bloque tiene réplicas almacenadas en otros DataNodes, permitiendo recuperar la información desde las copias disponibles.

2. ¿Qué ocurriría con un factor de replicación de 1?

Los bloques almacenados únicamente en el DataNode que falló dejarían de estar disponibles hasta recuperar ese nodo.

3. ¿Qué sucedería si fallaran dos DataNodes simultáneamente?

Podrían quedar bloques sin ninguna réplica disponible, provocando que algunos archivos no pudieran leerse completamente.

4. ¿Por qué replicación 2 no significa dos archivos completos en dos DataNodes?

Porque HDFS replica individualmente los bloques, que pueden distribuirse entre distintos DataNodes.

5. ¿Qué relación existe entre tamaño de bloque y número de bloques?

Para un archivo de tamaño fijo, cuanto menor sea el tamaño de bloque, mayor será la cantidad de bloques necesarios. Por el contrario, un tamaño de bloque mayor genera menos bloques.

6. ¿Por qué 10 GB representa mejor un sistema distribuido?

Porque el dataset de 10 GB contiene 75 bloques, mientras que el de 1 GB contiene únicamente 8. Esto permite observar una mayor distribución de datos y un proceso de recuperación más amplio ante la falla de un DataNode.

7. ¿Qué sucede con los bloques del DataNode que falló?

HDFS detecta los bloques subreplicados y utiliza las copias disponibles para generar nuevas réplicas en otros DataNodes, restableciendo el factor de replicación configurado.

8. ¿Quién administra la ubicación y replicación de los bloques?

El NameNode, que mantiene los metadatos del sistema y coordina la replicación.

9. ¿Qué diferencia hay entre disponibilidad y cumplimiento del factor de replicación?

Un archivo puede continuar disponible con una sola réplica de cada bloque, aunque temporalmente no cumpla el factor de replicación configurado.

7. Conclusiones

Las pruebas realizadas permitieron comprobar el funcionamiento del mecanismo de tolerancia a fallos de HDFS mediante la replicación de bloques.

Durante la caída de los DataNodes, ambos archivos permanecieron disponibles y pudieron leerse correctamente, demostrando la importancia del factor de replicación configurado en 2.

En el dataset de 1 GB se identificaron 3 bloques subreplicados de un total de 8, mientras que en el dataset de 10 GB se identificaron 45 bloques subreplicados de un total de 75.

Posteriormente, HDFS recuperó automáticamente las réplicas necesarias, restableciendo el estado de ambos archivos a HEALTHY, sin bloques subreplicados.

En conclusión, la replicación de HDFS permite mantener la disponibilidad de los datos ante la falla de un DataNode y recuperar la redundancia automáticamente cuando existen nodos disponibles con capacidad suficiente.